



1. What is the output of the program below? (10%)

```
int main() {
    bool k = 7;  $\leftarrow$  True
    for (int i = 8; i <= 12; i++) {  $8, 9, 10, 11, 12$ 
        if (i % 2 == 0) {
            continue;
        }
        for (int j = 3; j <= 9; j++) {  $3, 4, 5, 6, 7, 8, 9$ 
            if (j > 6) {
                break;  $\leftarrow$  先轉才除!
            }
            cout << (double)j/i << ", " << (double)(i/j) << endl;
             $\leftarrow$  先除完才轉!
        }
    }
    cout << (int)k << endl;
}
```

2. Write a simple C++ program to demonstrate how "short circuit evaluation" works. You should also explain your program. (10%)
3. Explain what is **call stack**. In addition, please describe the advantage and disadvantage of **inline function**. (10%)
4. Write a simple C++ program to demonstrate "function overloading". You should also explain your program. (10%)
5. Write a simple C++ program to demonstrate "function template". You should also explain your program. (10%)
6. What is the output of the following program? (5%) What problem does the following program have? You should explain the problem in detail. (5%)

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int p = 8;
5     int * q = new int;
6     int * r = new int;
7     *q = p;
8     *r = 6;
9     r = q;
10    *r += *q;
11    cout << *q << ", " << *r << endl;
12}
```

$$a = 8 \quad q + q = 16$$

$$r = 6 \quad q = q + q = 16$$

7. Explain what the difference between the **smart pointer** and the **raw pointer** is. (10%)
8. What is the output of the program below? (10%)

```
int * f(int c[3]) {
    c[2] = 6;
    return c;
}

int main() {
    int a[3] = {5, 2, 3};
    int *b;
    b = f(a);
    b[1] = a[0] + a[2];
    a[0] = *(b+1);
    for (int x: a) {
        cout << x << endl;
    }
}
```

$$b = [5, 11, 6]$$

$$a = [5, 11, 6]$$

9. Write three programs to demonstrate Call by Value, Call by Address, and Call by Reference in C++. You should explain the programs you write. (10%)

10.(1) Figure out the lines that will cause compilation error. (2) After removing the lines that will cause compilation error, what is the output of this program? (10%)

```

1 #include<iostream>
2 using namespace std;
3 class Midterm {
4 private:
5     int i1;
6     static int si1; =6
7 public:
8     int i2;
9     Midterm(int v) {
10         i1 = v+1;
11         i2 = v+2;
12         si1 = v+3;
13     }
14     static int si2; =8
15     static void setVariables(int x) {
16         X i1 = x+1;
17         si2 = x+2;
18     }
19 };
20
21 int Midterm::si1 = 6;
22 int Midterm::si2 = 8;
23
24 int main(int argc, const char * argv[]) {
25
26     Midterm midterm1(1);
27     Midterm midterm2(6);
28     midterm1.setVariables(8);
29
30     X cout << Midterm::si1 << endl;
31     cout << Midterm::si2 << endl;
32
33     midterm1.si2 = 7;
34
35     X cout << midterm1.i1 << endl;
36     X cout << midterm2.i1 << endl;
37     cout << midterm1.i2 << endl;
38     cout << midterm2.i2 << endl;
39     X cout << midterm1.si1 << endl;
40     X cout << midterm2.si1 << endl;
41     cout << midterm1.si2 << endl;
42     cout << midterm2.si2 << endl;
43 }

```

midterm1:
private:
int i1 2

public:
int i2 3

midterm2:
private:
int i1 7

public:
int i2 8

Static

private:
si1 = 8 & 9

public:
si2 = 8 & 7

9
10

2
3
4
5
6
7
8
9
7

Introduction to Programming (II) Midterm Exam

Name:

Student ID:

1. Write a simple C++ program to demonstrate "function overloading". You should also explain your program. (10%)
2. Write a simple C++ program to demonstrate "function template". You should also explain your program. (10%)
3. Write three programs to demonstrate **Call by Value**, **Call by Address**, and **Call by Reference** in C++. You should explain the programs you write. (10%)

Trace the given codes

1. Nested loops
2. Recursive functions
3. Pointers

2025 Exam

1、

```
int add(int a, int b){  
    return a+b; //1  
}
```

```
double add(double a, double b){  
    return a+b; //2  
}
```

```
int main(){
```

```
    cout << add(9, 18) << endl;
```

```
    cout << add(9.6, 18.6) << endl;
```

↖ call //1
↘ call //2

```
}
```

//1 的函式是兩個 int 相加，//2 的函式是兩個 double 相加。
雖然函式名稱都是 add，但傳遞的參數不同。

2、

```
template<typename T>  
T add(T a, T b){  
    return a+b;  
}
```

→ 這個 Template 在做兩數相加，

不過他可以接受任何形態，如 int, double

```
int main(){
```

```
    cout << add(2, 9);
```

```
    cout << add(2.6, 10.7);
```

```
}
```


3.

Call by Value · Address · Reference

```
void add_byVal (int a){  
    a += 1; // Call by Value  
}
```

```
void add_byAddr (int *a){  
    *a += 1;  
}
```

```
void add_byRef (int &a){  
    a += 1;  
}
```

```
int main(){  
    int x=10, y=10, z=10;  
    add_byVal(x);  
    add_byAddr(y);  
    add_byRef(z);  
    return 0;  
}
```

这三个函数都是將傳入的 Variables +1, 但實現方式与結果不同。

- add_byVal 是 Call by Value, 將变數複製後傳入函式, 不会改到原始值。
- add_byAddr 是 Call by Address, 將变數的記憶體位置直接傳給函式, 讓函式可以直接修改原变數。
- add_byRef 是 call by Reference, 將变數的參考傳給函式, 讓函式修改原变數。

2024 考古

1.

0.333333, 3
0.444444, 2
0.555555, 1
0.666666, 1
0.272727, 3
0.363636, 2
0.454545, 2
0.545455, 1

1

2.

```
#include <iostream>
using namespace std;
```

```
bool rF(){
    cout << "rF is called\n";
    return false;
}
```

```
bool rT(){
    cout << "rT is called\n";
    return true;
}
```

AND 的特性: 有 false 就一定 false

OR 的特性: 有 true 就一定 true

所以, 在 AND Test 時只有 rF 被呼叫, 在 OR Test 時只有 rT 被呼叫。

因為結果已確定, 剩下 2 個呼叫被短路。

```
int main(){
```

```
// AND Test
```

```
if (rF() && rT()){
```

```
    cout << "AND Result is T\n";
```

```
} else cout << "AND Result is F\n";
```

```
// OR Test
```

```
if (rT() || rF()){
```

```
    cout << "OR Result is T\n";
```

```
} else cout << "OR Result is F\n";
```

Output:

```
rF is called
AND Result is F
rT is called
OR Result is T
```

3.

Call stack 是一塊 LIFO 的記憶體區域，來管理函式呼叫的執行環境。每當執行一個函式，系統就 push 一個 Stack Frame，內含函式的參數、回傳位址、區域變數。
函式執行完後，則將其 pop 出此堆疊。

inline 的 advantage & disadvantage

advantage:

— 省去 Call Stack 與跳轉的時間成本，提升程式執行速度。

disadvantage:

— 每個呼叫點都會展開函式，使二進位檔變大、且會導致編譯時間變長。

4.

```
int add(int a, int b){  
    return a+b; //1  
}
```

```
double add(double a, double b){  
    return a+b; //2  
}
```

```
int main(){  
    cout << add(9, 18) << endl;  
    cout << add(9.6, 18.6) << endl;  
}
```

↗ call //1
↘ call //2

//1 的函式是兩個 int 相加，//2 的函式是兩個 double 相加。
雖然函式名稱都是 add，但傳遞的參數不同。

5.

```
template<typename T>
T add(T a, T b){
    return a+b;
}
```

→ 这个 Template 在做兩數相加，
不过他可以接受任何形態，如 int, double

```
int main(){
    cout << add(2, 9);
    cout << add(2.6, 10.7);
}
```

6.

16, 16

Line 9, 在 $r=q$ 前並未 delete r ,
导致 memory leaked.

7.

Raw Pointer: 需手动 new/delete, 易重複釋放或忘记釋放, 导致 memory Leaked.

Smart Pointer: 自动維護 reference count (shared ptr), 自动管理資源, 离佳開作用域後自动釋放, 安全性大幅提升。

8.

```
11 记得陣列是指標!
11
6
```

9.

Call by Value, Address, Reference

```
void add_byVal (int a){  
    a += 1; // Call by Value  
}  
  
void add_byAddr (int *a){  
    *a += 1;  
}  
  
void add_byRef (int &a){  
    a += 1;  
}  
  
int main(){  
    int x=10, y=10, z=10;  
    add_byVal(x);  
    add_byAddr(y);  
    add_byRef(z);  
    return 0;  
}
```

这三个函数都是將傳入的 Variables +1, 但實現方式与結果不同。

- add_byVal 是 Call by Value, 將變數複製後傳入函式, 不会改到原始值。
- add_byAddr 是 Call by Address, 將變數的記憶體位置直接傳給函式, 讓函式可以直接修改原變數。
- add_byRef 是 call by Reference, 將變數的參考傳給函式, 讓函式修改原變數。

10.

Line 16, static function 不能调用到 unstatic member variables.

Line 30, 35, 36, 39, 40 Private Variables 不可在函数外调用

10

3

8

7

7